

Decroly SANTANDER

UD3

API REST CON SPRING BOOT

MP_0613

Desarrollo Web en Entorno Servidor

3.2 API REST: la capa Controller

Qué vas a aprender en este tema

Ya creaste tus primeros endpoints: devolvían un texto y solo respondían a peticiones GET. En este tema vas a construir una **API REST** de verdad para la tienda: un servicio web que permite consultar, crear, modificar y borrar productos, que intercambia datos en JSON, que responde con los códigos de estado correctos y que informa de los errores de forma clara. Antes de programar, estudiarás qué es un servicio web, qué reglas sigue una API REST y cómo se representan los datos que viajan por la red.

1. Servicios web y arquitectura REST

Qué es un servicio web

Hasta ahora has programado aplicaciones web pensadas para **personas**: el servidor genera una página y un navegador la muestra. Un **servicio web** es distinto: es un componente de software que ofrece **datos u operaciones a otras aplicaciones** a través de la red, usando protocolos y formatos estándar de la web (HTTP, JSON, XML). Quien lo usa, el **consumidor** o cliente, es normalmente otro programa.

	Aplicación web tradicional	Servicio web
Quién lo usa	Una persona, con un navegador	Otra aplicación: una web JavaScript, una app móvil, otro servidor
Qué devuelve	Páginas HTML listas para ver	Datos estructurados (JSON o XML)
Qué le importa	El aspecto visual	La estructura y el significado de los datos

Los servicios web están por todas partes, aunque no los veas. Cuando pagas en una tienda online, la tienda llama al servicio web de la pasarela de pago. Cuando una app del tiempo muestra la previsión, consulta un servicio web (en España, por ejemplo, el de AEMET OpenData). Cuando una tienda te dice por dónde va tu paquete, consulta el servicio web de la empresa de transporte.

Ventajas de exponer la lógica de negocio como servicio web

Ventaja	Qué significa	En la tienda
Reutilización	La lógica se programa una vez y la usan todos los clientes.	La web, la app móvil y el panel de administración usan la misma API de productos.
Interoperabilidad	El cliente y el servidor pueden estar escritos en lenguajes distintos: solo tienen que hablar HTTP y JSON.	Nuestra API está en Java; la web que la consuma puede estar en JavaScript.
Desacoplamiento	El cliente no conoce el código del servidor, solo su interfaz. Se pueden cambiar por separado.	Se puede rediseñar la web sin tocar el backend.
Integración con terceros	Otras empresas pueden usar nuestros servicios, y nosotros los suyos.	Un comparador de precios podría consultar nuestro catálogo.

SOAP y REST

Existen dos grandes formas de construir servicios web:

	SOAP	REST
Qué es	Un **protocolo** con reglas estrictas	Un **estilo de arquitectura**: un conjunto de principios
Formato	Siempre XML, dentro de un "sobre" (*envelope*)	Normalmente JSON; también XML u otros
Contrato	Obligatorio: un documento WSDL describe cada operación	Opcional: se suele documentar con OpenAPI.
Uso de HTTP	Casi todo va por POST a una única dirección	Aprovecha HTTP: métodos, rutas y códigos de estado
Dónde se usa	Banca, administración pública, sistemas antiguos	La gran mayoría de las APIs web y móviles actuales

Existen otras alternativas más recientes, como **GraphQL** (el cliente pide exactamente los campos que necesita) o **gRPC** (comunicación binaria muy rápida entre servidores), pero REST sigue siendo el estándar de hecho, y es el que usaremos.

Los principios de REST

REST (*Representational State Transfer*) organiza una API alrededor de **recursos**. Estos son sus principios:

Principio	Qué significa	En la tienda
Recursos identificados por URI	Todo lo que se puede consultar o modificar es un recurso con su propia dirección.	`/api/productos` es la colección de productos; `/api/productos/7` es el producto 7.
Interfaz uniforme	Las operaciones sobre un recurso se expresan con los métodos HTTP estándar, no inventando rutas.	`DELETE /api/productos/7` borra, en lugar de `/api/borrarProducto?id=7`.
Representaciones	El cliente no recibe el recurso, sino una representación suya en un formato.	El producto 7 se envía como JSON o como XML.
Sin estado	Cada petición lleva toda la información necesaria. El servidor no recuerda peticiones anteriores del cliente.	Para ver el producto 7 siempre se pide `/api/productos/7`; no existe "el producto que miré antes".
Cliente-servidor	Cliente y servidor evolucionan por separado.	El frontend y la API son proyectos distintos.

Cómo diseñar las URI

Regla	Correcto	Incorrecto
Sustantivos en plural, no verbos	`GET /api/productos`	`GET /api/obtenerProductos`
La acción la indica el método HTTP	`DELETE /api/productos/7`	`GET /api/productos/7/borrar`
El identificador va en la ruta	`/api/productos/7`	`/api/productos?id=7`
Filtros y búsquedas como parámetros	`/api/productos?maxPrecio=20`	`/api/productosBaratos`
Jerarquía para recursos relacionados	`/api/familias/3/productos`	`/api/productosDeFamilia/3`

Regla	Correcto	Incorrecto
Minúsculas y guiones	`/api/formas-pago`	`/api/FormasPago`

Es habitual agrupar toda la API bajo el prefijo `/api` para distinguirla de otras rutas de la aplicación. Algunas APIs añaden también una versión (`/api/v1/productos`) para poder cambiar la API sin romper a los clientes antiguos.

Esta es la API de productos que construirás en este tema:

Método	URI	Acción	Código si va bien
GET	`/api/productos`	Listar todos los productos	200 OK
GET	`/api/productos/{id}`	Consultar un producto	200 OK
POST	`/api/productos`	Crear un producto	201 Created
PUT	`/api/productos/{id}`	Modificar un producto	200 OK
DELETE	`/api/productos/{id}`	Borrar un producto	204 No Content

2. Formatos de intercambio de datos

JSON

JSON (*JavaScript Object Notation*) es el formato más usado en las APIs REST. Es texto plano, fácil de leer para las personas y de procesar para los programas. Solo tiene dos estructuras: el **objeto** (entre llaves, pares `"clave": valor`) y el **array** (entre corchetes, lista de valores).

Un producto en JSON

```
{
  "id": 7,
  "nombre": "Teclado mecanico",
  "precio": 49.9,
  "stock": 15,
  "disponible": true,
  "etiquetas": ["gaming", "usb"],
  "familia": { "id": 2, "nombre": "Perifericos" },
  "descuento": null
}
```

Tipo JSON	Ejemplo	Tipo Java equivalente
Cadena	`"Teclado"` (siempre comillas dobles)	`String`
Número	`49.9`, `15`	`int`, `long`, `double`…
Booleano	`true`, `false`	`boolean`
Nulo	`null`	`null`
Objeto	`{ "id": 2, … }`	Un objeto de una clase
Array	`[ … ]`	`List`, array

ATENCIÓN · Errores típicos al escribir JSON a mano

- Las claves van **siempre** entre comillas dobles: `{ nombre: "Ana" }` no es válido.
- No se admiten comillas simples ni una coma detrás del último elemento.
- No hay comentarios.

Un JSON mal escrito en una petición provoca un error **400 Bad Request**.

XML

XML es el formato clásico de intercambio y el de SOAP. Sigue usándose en sistemas empresariales y administraciones. El mismo producto en XML:

Un producto en XML

```
<producto>
    <id>7</id>
    <nombre>Teclado mecanico</nombre>
    <precio>49.9</precio>
    <stock>15</stock>
</producto>
```

	JSON	XML
Tamaño	Más compacto	Más largo (etiquetas de apertura y cierre)
Tipos de datos	Distingue números, textos, booleanos	Todo es texto; los tipos se definen aparte (XSD)
Uso en navegador	Nativo en JavaScript	Requiere procesarlo
Uso típico	APIs REST, aplicaciones web y móviles	SOAP, sistemas empresariales, ficheros de configuración

De objeto Java a JSON: Jackson

Dentro de tu aplicación los datos son **objetos Java**; por la red viajan como **texto JSON**. La conversión la hace automáticamente **Jackson**, la librería que viene incluida en el starter web:

- **Serialización:** convertir un objeto Java en JSON. Ocurre cuando tu método del controlador devuelve un objeto.
- **Deserialización:** convertir un JSON en un objeto Java. Ocurre cuando el cliente envía datos en el cuerpo de la petición.

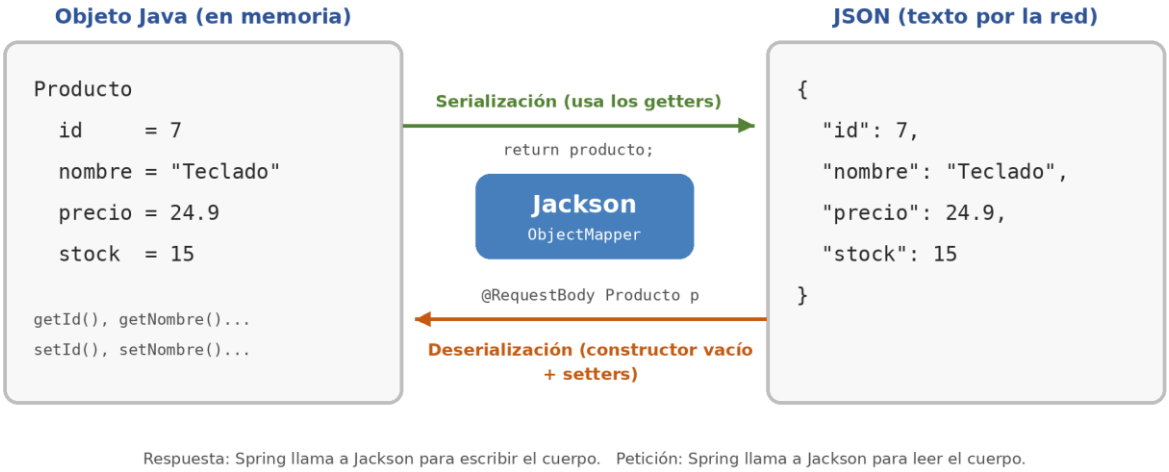


Figura 1. Jackson convierte objetos Java en JSON y JSON en objetos Java.

La clase de dominio: un POJO

Para representar un producto necesitamos una clase. Será un **POJO** (*Plain Old Java Object*): una clase Java sencilla, sin herencias especiales, con:

- **Atributos privados**, uno por cada dato.
- Un **constructor vacío**: Jackson lo usa para crear el objeto al deserializar.
- **Getters y setters**: Jackson usa los getters para leer los atributos al serializar y los setters para rellenarlos al deserializar. El nombre de cada clave JSON sale del getter: `getPrecio()` produce la clave `"precio"`.

Práctica 1. Primeros endpoints de productos

1. En tienda-decroly, crea el paquete `es.decroly.tienda.domain` y dentro la clase `Producto`:

Producto.java

```
package es.decroly.tienda.domain;

public class Producto {

    private Long id;
    private String nombre;
    private double precio;
    private int stock;

    public Producto() {
    }

    public Producto(Long id, String nombre, double precio, int stock) {
        this.id = id;
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public double getPrecio() { return precio; }
    public void setPrecio(double precio) { this.precio = precio; }

    public int getStock() { return stock; }
    public void setStock(int stock) { this.stock = stock; }
}
```

Puedes generar los getters y setters con IntelliJ: Alt+Insert › **Getter and Setter** y selecciona todos los atributos.

1. En el paquete `controllers`, crea `ProductoRestController` con una lista de productos de prueba y los dos endpoints de consulta:

ProductoRestController.java (versión 1)

```
package es.decroly.tienda.controllers;

import es.decroly.tienda.domain.Producto;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicLong;

@RestController
public class ProductoRestController {
```

```
    // PROVISIONAL: los datos viven en memoria (ver recuadro)
    private final List<Producto> productos = new ArrayList<>();
    private final AtomicLong secuencia = new AtomicLong();

    public ProductoRestController() {
        anadir("Teclado mecanico", 49.90, 15);
        anadir("Raton inalambrico", 19.95, 40);
        anadir("Monitor 24 pulgadas", 139.00, 8);
    }

    private void anadir(String nombre, double precio, int stock) {
        Long id = secuencia.incrementAndGet();
        productos.add(new Producto(id, nombre, precio, stock));
    }

    @GetMapping("/api/productos")
    public List<Producto> listar() {
        return productos;
    }

    @GetMapping("/api/productos/{id}")
    public Producto buscarPorId(@PathVariable Long id) {
        return productos.stream()
                .filter(p -> p.getId().equals(id))
                .findFirst()
                .orElse(null);
        /*Sin usar stream:
        for (Producto p : productos) {
                if (p.getId().equals(id)) {
                    return p;
                }
            }
            return null;
        */
    }
}
```

Elemento	Explicación
`List<Producto> productos`	Hace de "base de datos" provisional. Al devolver la lista, Jackson la convierte en un array JSON.
`AtomicLong secuencia`	Genera identificadores consecutivos (1, 2, 3…), como haría una columna autoincremental de MySQL.
Constructor	Spring crea **un único** objeto de cada controlador (un bean) al arrancar, así que el constructor se ejecuta una sola vez y carga los datos de prueba.
`stream().filter(…).findFirst()`	Busca el primer producto cuyo id coincide. `orElse(null)` devuelve `null` si no lo encuentra.

1. Arranca la aplicación y abre `http://localhost:8080/api/productos` en el navegador. Comprueba que recibes un array JSON con tres objetos.

2. Abre `http://localhost:8080/api/productos/2`. Recibes un solo objeto.
3. Abre `http://localhost:8080/api/productos/99` y mira en F12 › Red el código de estado. Recibes un **200 OK con el cuerpo vacío**. ¿Es correcto que un producto que no existe devuelva "todo bien"?.
4. Añade a `Producto` un getter nuevo: `public double getPrecioConIva() { return precio * 1.21; }`. Reinicia y consulta un producto. ¿Qué ha aparecido en el JSON? ¿Por qué?

ATENCIÓN · Solución provisional: datos dentro del controlador

Guardar los datos en una lista dentro del controlador **no es correcto** y solo lo hacemos para empezar. Tiene tres problemas:

- **Rompe la arquitectura por capas:** el controlador está haciendo de repositorio. En el tema 3.3 moverás los datos y la lógica a un servicio.
- **Los datos se pierden** cada vez que reinicias la aplicación. Mas adelante los guardarás en MySQL.
- `ArrayList` **no es segura** si llegan varias peticiones a la vez, porque cada petición se atiende en un hilo distinto. Con una base de datos este problema desaparece.

3. Cómo probar una API

El navegador solo sabe enviar peticiones **GET** al escribir una dirección. Para probar una API necesitas una herramienta que permita elegir el método HTTP, añadir cabeceras y enviar un cuerpo JSON. Estas son las más habituales:

Herramienta	Tipo	Comentario
Postman	Aplicación de escritorio	La más extendida. Permite guardar las peticiones en colecciones y compartirlas. Para guardar colecciones pide una cuenta gratuita.
curl	Línea de comandos	Viene instalado en Windows 10/11, Linux y macOS. Ideal para pruebas rápidas y scripts.
Bruno	Aplicación de escritorio	Alternativa libre a Postman que guarda las colecciones como ficheros del proyecto.
HTTP Client de IntelliJ	Integrado en el IDE	Disponible en la edición Ultimate: peticiones escritas en ficheros `.http`.

Conceptos de Postman

Concepto	Qué es
Request	Una petición guardada: método, URL, cabeceras y cuerpo.
Collection	Una carpeta de peticiones relacionadas. Crearemos una colección `tienda-decroly`. Se puede exportar a un fichero JSON y compartir.
Environment	Un conjunto de variables. Crearemos el entorno `local` con la variable `baseUrl`.
Variable	Se escribe entre dobles llaves: `{{baseUrl}}/api/productos`. Si mañana la API cambia de dirección, solo cambias la variable.

Práctica 2. Colección de Postman para tienda-decroly

1. Descarga Postman desde `https://www.postman.com/downloads/` e instálalo. Crea una cuenta gratuita cuando te lo pida.
2. En **Environments**, crea el entorno `local` con la variable `baseUrl` y el valor `http://localhost:8080`. Selecciónalo en el desplegable de entornos (arriba a la derecha).
3. En **Collections**, crea la colección `tienda-decroly` y dentro una carpeta `Productos`.
4. Añade la petición **Listar productos**: método GET, URL `{{baseUrl}}/api/productos`. Pulsa **Send** y observa el cuerpo, el código de estado y el tiempo de respuesta.
5. Añade **Consultar producto** (`{{baseUrl}}/api/productos/1`) y **Consultar producto inexistente** (`{{baseUrl}}/api/productos/99`). Guárdalas con Ctrl+S.
6. Revisa la pestaña **Headers** de la respuesta y localiza `Content-Type`.

Las mismas pruebas con curl

Abre una terminal. En Windows usa `cmd` y escribe `curl.exe` (en PowerShell, `curl` a secas es otro comando distinto). La opción `-i` muestra también el código de estado y las cabeceras:

Terminal (cmd)

```
curl.exe -i http://localhost:8080/api/productos
curl.exe -i http://localhost:8080/api/productos/99
```

4. Los métodos HTTP en REST

Cada método HTTP tiene un significado. Respetarlo es lo que hace que cualquier programador entienda tu API sin leer su código.

Método	Significado	¿Lleva cuerpo?	¿Seguro?	¿Idempotente?
GET	Consultar	No	Sí	Sí
POST	Crear un recurso nuevo	Sí	No	No
PUT	Reemplazar un recurso completo	Sí	No	Sí
PATCH	Modificar parte de un recurso	Sí	No	Depende
DELETE	Borrar	No	No	Sí

- Un método es **seguro** si no modifica nada en el servidor. Un GET se puede repetir, cachear o precargar sin miedo.
- Un método es **idempotente** si repetirlo tiene el mismo efecto que hacerlo una vez. Borrar el producto 7 dos veces deja la tienda igual que borrarlo una. En cambio, enviar dos veces el mismo POST crea **dos** productos. Por eso los navegadores avisan antes de reenviar un formulario.

Recibir datos en el cuerpo: @RequestBody

Anteriormente recibías datos pequeños en la URL. Para crear o modificar un producto hay que enviar el objeto completo en el **cuerpo** de la petición, en JSON. La anotación `@RequestBody` indica a Spring que debe deserializar el cuerpo y convertirlo en un objeto Java:

Uso de @RequestBody

```
@PostMapping("/api/productos")
public Producto crear(@RequestBody Producto producto) {
    // 'producto' ya es un objeto Java con los datos del JSON
}
```

El cliente debe indicar el formato del cuerpo con la cabecera `Content-Type: application/json`. Si no lo hace, Spring no sabe cómo leerlo y responde **415 Unsupported Media Type**.

Rutas comunes: @RequestMapping en la clase

Todos los endpoints de productos empiezan por `/api/productos`. Para no repetirlo, se coloca `@RequestMapping` sobre la clase y cada método indica solo la parte final:

Esquema (no es código completo)

```
@RestController
@RequestMapping("/api/productos")
public class ProductoRestController {

    @GetMapping              // GET    /api/productos
    @GetMapping("/{id}")     // GET    /api/productos/{id}
    @PostMapping             // POST   /api/productos
    @PutMapping("/{id}")     // PUT    /api/productos/{id}
    @DeleteMapping("/{id}")  // DELETE /api/productos/{id}
}
```

Práctica 3. CRUD completo de productos

Modifica `ProductoRestController` para añadir `@RequestMapping` en la clase y completar las operaciones. Fíjate en el método privado `buscar`: devuelve un `Optional`, un contenedor que puede tener un producto o estar vacío, y evita repetir la búsqueda en cada método.

ProductoRestController.java (versión 2)

```
@RestController
@RequestMapping("/api/productos")
public class ProductoRestController {

    // ... lista, secuencia y constructor igual que en la version 1 ...

    @GetMapping
    public List<Producto> listar() {
        return productos;
    }

    @GetMapping("/{id}")
    public Producto buscarPorId(@PathVariable Long id) {
        return buscar(id).orElse(null);
    }

    @PostMapping
    public Producto crear(@RequestBody Producto producto) {
        producto.setId(secuencia.incrementAndGet());
        productos.add(producto);
        return producto;
    }

    @PutMapping("/{id}")
    public Producto modificar(@PathVariable Long id,
                              @RequestBody Producto datos) {
        Optional<Producto> existente = buscar(id);
        if (existente.isEmpty()) {
            return null;
        }
        Producto producto = existente.get();
        producto.setNombre(datos.getNombre());
        producto.setPrecio(datos.getPrecio());
        producto.setStock(datos.getStock());
        return producto;
```

```
    }

    @DeleteMapping("/{id}")
    public void borrar(@PathVariable Long id) {
        productos.removeIf(p -> p.getId().equals(id));
    }

    private Optional<Producto> buscar(Long id) {
        return productos.stream()
                .filter(p -> p.getId().equals(id))
                .findFirst();
    }
}
```

Importa `java.util.Optional` y las anotaciones `PostMapping`, `PutMapping`, `DeleteMapping`, `RequestBody` y `RequestMapping` (Alt+Enter).

Detalle	Por qué
En `crear`, el id lo asigna el servidor	El cliente no decide los identificadores. Aunque envíe un `id` en el JSON, se sobrescribe.
En `modificar`, el id sale de la URL	La URI identifica el recurso que se modifica; el cuerpo solo trae los datos nuevos.
`borrar` devuelve `void`	No hay nada que devolver. Spring responde sin cuerpo.

Pruébalo en Postman

1. Crea la petición **Crear producto**: método POST, URL `{{baseUrl}}/api/productos`. En **Body** elige **raw** y, en el desplegable de la derecha, **JSON**. Postman añadirá la cabecera `Content-Type` automáticamente.

Cuerpo de la petición POST

```
{
  "nombre": "Alfombrilla XL",
  "precio": 12.50,
  "stock": 60
}
```

1. Envíala dos veces. ¿Qué id recibe cada producto? Consulta el listado. (Recuerda: POST no es idempotente.)
2. Crea **Modificar producto** (PUT a `{{baseUrl}}/api/productos/1`) con un cuerpo que cambie el precio. Comprueba el cambio con un GET.
3. Crea **Borrar producto** (DELETE a `{{baseUrl}}/api/productos/2`) y compruébalo con el listado.

4. Reinicia la aplicación y vuelve a listar. ¿Qué ha pasado con los cambios?
5. Envía el POST con un JSON mal escrito (por ejemplo, quita unas comillas) y anota el código de estado. Después, cambia el tipo del cuerpo a **Text** y anota el código que obtienes.

La misma creación con curl (en `cmd`, las comillas interiores del JSON se escapan con `\`):

Terminal (cmd)

```
curl.exe -i -X POST http://localhost:8080/api/productos ^
  -H "Content-Type: application/json" ^
  -d "{\"nombre\":\"Alfombrilla XL\",\"precio\":12.5,\"stock\":60}"
```

5. Códigos de estado y ResponseEntity

El código de estado dice cómo ha ido la operación

Cada respuesta HTTP empieza con un número de tres cifras: el **código de estado**. El cliente lo mira **antes** de leer el cuerpo, para saber qué ha pasado. La primera cifra indica el tipo de resultado: **2** si todo ha ido bien, **4** si la petición del cliente es incorrecta y **5** si ha fallado el servidor.

Familia	Significado	Códigos más usados en una API

2xx	Éxito	`200 OK`
		`201 Created`
		`204 No Content`

3xx	Redirección	`301 Moved Permanently` `304 Not Modified`

4xx	Error del **cliente**: la petición es incorrecta	`400 Bad Request`
		`401 Unauthorized`
		`403 Forbidden`
		`404 Not Found`
		`405 Method Not Allowed`
		`409 Conflict`
		`415 Unsupported Media Type`

5xx	Error del **servidor**: la petición era correcta pero algo falló	`500 Internal Server Error` `503 Service Unavailable`

La diferencia entre 4xx y 5xx es importante: un 4xx le dice al cliente "corrige tu petición"; un 5xx le dice "el problema es nuestro, reintenta más tarde".

Aplicado a nuestra API de productos, esto es lo que debe responder cada operación (los datos incorrectos:

Petición	Si va bien	Si el producto no existe
`GET /api/productos`	200 y la lista (vacía si no hay productos)	—
`GET /api/productos/{id}`	200 y el producto	404
`POST /api/productos`	201 y el producto creado	—
`PUT /api/productos/{id}`	200 y el producto modificado	404
`DELETE /api/productos/{id}`	204, sin cuerpo	404

La cabecera Location

Cuando se crea un recurso, la respuesta 201 debe incluir la cabecera `Location` con la URI del recurso nuevo, por ejemplo `Location: http://localhost:8080/api/productos/4`. Así el cliente sabe dónde consultarlo sin tener que construir la dirección.

ResponseEntity: la respuesta completa

Hasta ahora, tus métodos devuelven solo los **datos** (un producto, una lista) y Spring añade siempre el código 200. Para elegir el código, el método tiene que devolver la **respuesta completa**. En Spring, una respuesta completa es un objeto de la clase `ResponseEntity`.

Una respuesta HTTP es como una carta

- El **código de estado** es el sello que dice si es una buena o una mala noticia.
- Las **cabeceras** son lo que va escrito en el sobre: datos sobre el envío, como el formato del contenido.
- El **cuerpo** es la carta en sí: los datos. Puede ir vacío.

Cuando tu método devuelve un `Producto`, solo entregas la carta y Spring pone siempre el sello del 200. Con `ResponseEntity` preparas tú el **sobre completo**: sello, sobre y carta.

Entre `< >` se indica qué lleva dentro el cuerpo: `ResponseEntity<Producto>` lleva un producto y `ResponseEntity<Void>` (con mayúscula) no lleva nada. Para crear la respuesta no se usa `new`, sino estos métodos ya preparados:

Escribes	Respuesta que se envía
`ResponseEntity.ok(producto)`	200 con el producto en el cuerpo

Escribes	Respuesta que se envía
`ResponseEntity.created(direccion).body(producto)`	201 con el producto y la cabecera `Location`
`ResponseEntity.noContent().build()`	204 sin cuerpo
`ResponseEntity.notFound().build()`	404 sin cuerpo

Una regla para recordarlos: los que terminan en `build()` crean una respuesta **vacía**; los que llevan `ok(…)` o `body(…)` llevan **datos**.

Práctica 4. Respuestas con el código correcto

No vas a cambiar todo el controlador de golpe. Modifica los métodos **de uno en uno** y prueba cada cambio en Postman antes de pasar al siguiente. El método `buscar` **no cambia**: solo cambia lo que hacemos con su resultado.

Paso 1. Consultar un producto: 200 o 404

ProductoRestController.java: buscarPorId

```
@GetMapping("/{id}")
public ResponseEntity<Producto> buscarPorId(@PathVariable Long id) {
    Producto producto = buscar(id);
    if (producto == null) {
        return ResponseEntity.notFound().build();     // 404
    }
    return ResponseEntity.ok(producto);               // 200
}
```

Importa `ResponseEntity` de `org.springframework.http` (Alt+Enter). Compara con la versión anterior: antes, si el producto no existía, devolvíamos `null` y el cliente recibía un 200 vacío. Ahora le decimos claramente que no existe.

1. Reinicia la aplicación y lanza en Postman **Consultar producto**: sigue respondiendo 200 con el producto.
2. Lanza **Consultar producto inexistente**: ahora responde **404**. Fíjate en el código, arriba a la derecha de la respuesta.

Paso 2. Borrar: 204 o 404

ProductoRestController.java: borrar

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> borrar(@PathVariable Long id) {
    Producto producto = buscar(id);
    if (producto == null) {
        return ResponseEntity.notFound().build();     // 404
    }
    productos.remove(producto);
    return ResponseEntity.noContent().build();        // 204
}
```

1. Borra el producto 2 en Postman: responde **204** y el cuerpo está vacío.
2. Vuelve a enviar exactamente la misma petición. Ahora responde **404**. ¿Por qué ha cambiado la respuesta si la petición es la misma?

Paso 3. Modificar: 200 o 404

Hazlo tú antes de mirar la solución: sigue el mismo patrón que en el paso 1.

Solución: modificar

```
@PutMapping("/{id}")
public ResponseEntity<Producto> modificar(@PathVariable Long id,
                                          @RequestBody Producto datos) {
    Producto producto = buscar(id);
    if (producto == null) {
        return ResponseEntity.notFound().build();     // 404
    }
    producto.setNombre(datos.getNombre());
    producto.setPrecio(datos.getPrecio());
    producto.setStock(datos.getStock());
    return ResponseEntity.ok(producto);               // 200
}
```

Paso 4. Crear: 201 y la cabecera Location

Cuando se crea algo nuevo, la respuesta correcta es **201 Created**. Además, debe incluir la cabecera `Location` con la **dirección del producto creado**, para que el cliente sepa dónde consultarlo.

ProductoRestController.java: crear

```
@PostMapping
public ResponseEntity<Producto> crear(@RequestBody Producto producto) {
    producto.setId(secuencia.incrementAndGet());
    productos.add(producto);
    URI direccion = URI.create("/api/productos/" + producto.getId());
    return ResponseEntity.created(direccion).body(producto);   // 201
}
```

`URI` es la clase de Java que representa una dirección; impórtala de `java.net`. `URI.create` la construye a partir de un texto: si el producto creado tiene el id 4, la cabecera será `Location: /api/productos/4`.

1. Envía **Crear producto** en Postman: responde **201**.
2. Abre la pestaña **Headers** de la respuesta y localiza `Location`. Haz un GET a esa dirección: debe devolver el producto que acabas de crear.

Paso 5. Listar

`listar` puede quedarse como está: siempre responde 200 con la lista. Si no hay productos, devuelve una lista vacía `[]`, que también es una respuesta correcta: la colección existe, aunque esté vacía.

Completa la colección de Postman

Añade a la colección las peticiones **Modificar producto inexistente** y **Borrar producto inexistente**, y comprueba que todas las peticiones responden con el código de la tabla del vista anteriormente.

El patrón que se repite

Fíjate en que este bloque aparece, casi igual, en tres métodos:

```
Producto producto = buscar(id);
if (producto == null) {
    return ResponseEntity.notFound().build();
}
```

Con veinte endpoints tendrías veinte copias. En el apartado siguiente aprenderás a escribirlo en un solo sitio.

6. Errores con excepciones

El problema

Tras la Práctica 4, tu API responde con los códigos correctos, pero le quedan dos defectos:

- **Repite código:** el mismo `if (producto == null)` aparece en tres métodos.
- **Los errores no explican nada:** el 404 llega con el cuerpo vacío. El cliente sabe que algo ha fallado, pero no qué. Y todavía no comprobamos los datos: se puede crear un producto con el precio negativo.

La solución se apoya en algo que ya conoces de Java: las **excepciones**. La resolveremos en dos escalones: en este apartado, harás que tu código **avise** del error lanzando una excepción.

Repaso: lanzar una excepción

Cuando un método encuentra un problema que no puede resolver, **lanza** una excepción con `throw`. En ese momento la ejecución del método **se corta**: no se ejecuta ninguna línea más, y la excepción "sube" al método que lo había llamado.

Lanzar una excepción

```
if (precio < 0) {
    throw new IllegalArgumentException("Precio negativo");
}
// si se ha lanzado la excepcion, esta linea ya no se ejecuta
```

¿Y qué pasa si la excepción sale de un método del controlador? La recibe **Spring**, que es quien había llamado a ese método. Vamos a aprovecharlo: crearemos nuestras propias excepciones y le diremos a Spring qué código debe responder con cada una.

Excepciones propias con @ResponseStatus

Crearemos una excepción para cada tipo de error de la tienda. Son clases muy cortas que heredan de `RuntimeException` (así no hay que declararlas con `throws` en cada método) y solo reciben el mensaje que explica el error:

Excepción	Se lanza cuando…	Código
`RecursoNoEncontradoException`	Se pide un producto que no existe.	404
`PeticionIncorrectaException`	Los datos recibidos no son válidos.	400

La anotación `@ResponseStatus`, puesta sobre la clase de la excepción, le dice a Spring qué código debe responder cuando la reciba:

Una excepción que sabe su código

```
@ResponseStatus(HttpStatus.NOT_FOUND)                  // 404
public class RecursoNoEncontradoException extends RuntimeException {

    public RecursoNoEncontradoException(String mensaje) {
        super(mensaje);       // RuntimeException guarda el mensaje
    }
}
```

Ahora el método `buscar` puede **lanzar** la excepción en lugar de devolver `null`. Lo llamaremos `buscarOLanzar`, para que su nombre avise de lo que hace:

buscarOLanzar

```
private Producto buscarOLanzar(Long id) {
    for (Producto p : productos) {
        if (p.getId().equals(id)) {
            return p;
        }
    }
    throw new RecursoNoEncontradoException("No existe el producto " + id);
}
```

Y aquí está la gran ventaja: los métodos del controlador **ya no necesitan el `if`**. Si `buscarOLanzar` devuelve algo, el producto existe; si no existe, la excepción se lanza y el resto del método no llega a ejecutarse:

buscarPorId, ahora sin if

```
@GetMapping("/{id}")
public Producto buscarPorId(@PathVariable Long id) {
    return buscarOLanzar(id);    // si no existe, no llega aqui
}
```

El método vuelve a devolver un simple `Producto`: si llega al `return`, todo ha ido bien y el 200 es correcto.

Práctica 5. Lanza excepciones desde el controlador

1. Crea el paquete `es.decroly.tienda.exceptions` y dentro las dos excepciones:

RecursoNoEncontradoException.java

```
package es.decroly.tienda.exceptions;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.NOT_FOUND)
public class RecursoNoEncontradoException extends RuntimeException {
    public RecursoNoEncontradoException(String mensaje) {
        super(mensaje);
    }
}
```

PeticionIncorrectaException.java

```
package es.decroly.tienda.exceptions;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.BAD_REQUEST)
public class PeticionIncorrectaException extends RuntimeException {
    public PeticionIncorrectaException(String mensaje) {
        super(mensaje);
    }
}
```

2. En el controlador, sustituye el método `buscar` por `buscarOLanzar` y añade un método `validar` que compruebe los datos recibidos:

ProductoRestController.java: métodos privados

```
private Producto buscarOLanzar(Long id) {
    for (Producto p : productos) {
        if (p.getId().equals(id)) {
            return p;
        }
    }
    throw new RecursoNoEncontradoException("No existe el producto " + id);
}

private void validar(Producto p) {
    if (p.getNombre() == null || p.getNombre().isBlank()) {
        throw new PeticionIncorrectaException("El nombre es obligatorio");
    }
    if (p.getPrecio() < 0) {
        throw new PeticionIncorrectaException("Precio negativo");
    }
}
```

`isBlank()` devuelve `true` si el texto está vacío o solo tiene espacios.

3. Reescribe los cuatro métodos que usaban `buscar`. Compara cada uno con su versión de la Práctica 4: los `if` han desaparecido.

ProductoRestController.java: métodos públicos

```
@GetMapping("/{id}")
public Producto buscarPorId(@PathVariable Long id) {
    return buscarOLanzar(id);
}

@PostMapping
public ResponseEntity<Producto> crear(@RequestBody Producto producto) {
    validar(producto);                          // puede lanzar un 400
    producto.setId(secuencia.incrementAndGet());
    productos.add(producto);
    URI direccion = URI.create("/api/productos/" + producto.getId());
    return ResponseEntity.created(direccion).body(producto);
```

```
}

@PutMapping("/{id}")
public Producto modificar(@PathVariable Long id,
                          @RequestBody Producto datos) {
    validar(datos);                              // puede lanzar un 400
    Producto producto = buscarOLanzar(id);       // puede lanzar un 404
    producto.setNombre(datos.getNombre());
    producto.setPrecio(datos.getPrecio());
    producto.setStock(datos.getStock());
    return producto;
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> borrar(@PathVariable Long id) {
    Producto producto = buscarOLanzar(id);       // puede lanzar un 404
    productos.remove(producto);
    return ResponseEntity.noContent().build();
}
```

`crear` y `borrar` siguen devolviendo `ResponseEntity`, porque sus respuestas correctas (201 y 204) no son un 200.

1. Reinicia y repite en Postman **Consultar producto inexistente**: sigue respondiendo 404, aunque el método ya no tiene ningún `if`.
2. Crea un producto con el nombre vacío (`"nombre": ""`) y otro con el precio negativo. Ambos deben responder **400**.
3. Mira el **cuerpo** de esas respuestas de error. Spring añade un JSON con la hora, el código y la ruta, pero ¿aparece tu mensaje, por ejemplo "No existe el producto 99"?.

> **ATENCIÓN · La validación todavía no está en su sitio**
>
> Comprobar que el precio no es negativo es una **regla de negocio**, así que pertenece a la capa de servicios. Más adelante la moverás allí y, más adelante todavía, la sustituirás por anotaciones de validación (`@NotBlank`, `@PositiveOrZero`…).

7. Un manejador global de errores

Un único sitio que responde a los errores

`@ResponseStatus` decide el **código**, pero no el **cuerpo** de la respuesta, y por eso tu mensaje no llega al cliente. Para controlar la respuesta completa de un error, Spring permite crear una clase que **atiende las excepciones de todos los controladores**. Se marca con `@RestControllerAdvice` y funciona como un servicio de atención de errores:

- Cada método de esa clase se marca con `@ExceptionHandler(…)` e indica **qué excepción atiende**.
- Cuando cualquier controlador lanza esa excepción, Spring llama a ese método y le pasa la excepción.
- Lo que devuelve el método es la respuesta que recibe el cliente.

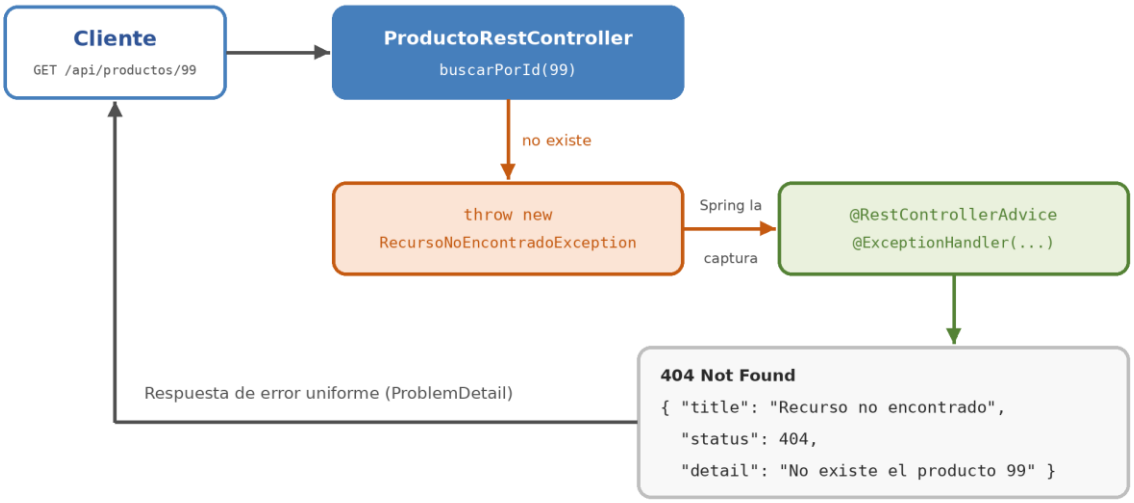


Figura 2. Una excepción lanzada en el controlador se convierte en una respuesta de error en el manejador global.

ProblemDetail: un formato estándar para los errores

¿Qué ponemos en el cuerpo de un error? En lugar de inventar un formato propio, usaremos el que propone el estándar HTTP para describir errores, y que Spring ofrece con la clase `ProblemDetail`. Una respuesta de error tendrá este aspecto:

Respuesta de error con ProblemDetail

```
{
  "title": "Recurso no encontrado",
  "status": 404,
  "detail": "No existe el producto 99"
}
```

Campo	Qué contiene	De dónde sale
`title`	El tipo de error, en pocas palabras.	Lo escribes tú en el manejador.
`status`	El código de estado.	Lo indicas tú en el manejador.
`detail`	Qué ha pasado exactamente esta vez.	Es el mensaje que escribiste en el `throw`.

Spring añade además dos campos, `type` e `instance` (la ruta de la petición), que de momento puedes ignorar.

Práctica 6. Errores centralizados con ProblemDetail

4. En el paquete `exceptions`, crea el manejador global:

ManejadorGlobalErrores.java

```
package es.decroly.tienda.exceptions;

import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class ManejadorGlobalErrores {

    @ExceptionHandler(RecursoNoEncontradoException.class)
    public ProblemDetail noEncontrado(RecursoNoEncontradoException ex) {
        ProblemDetail problema = ProblemDetail.forStatusAndDetail(
                HttpStatus.NOT_FOUND, ex.getMessage());
        problema.setTitle("Recurso no encontrado");
        return problema;
    }

    @ExceptionHandler(PeticionIncorrectaException.class)
    public ProblemDetail incorrecta(PeticionIncorrectaException ex) {
        ProblemDetail problema = ProblemDetail.forStatusAndDetail(
                HttpStatus.BAD_REQUEST, ex.getMessage());
        problema.setTitle("Peticion incorrecta");
        return problema;
    }
}
```

Línea	Qué hace
`@RestControllerAdvice`	Convierte la clase en el manejador de errores de **todos** los controladores.
`@ExceptionHandler(RecursoNoEncontradoException.class)`	Este método se ejecuta cuando cualquier controlador lanza esa excepción.
Parámetro `ex`	Es la excepción que se lanzó. `ex.getMessage()` devuelve el texto que escribiste en el `throw`.
`ProblemDetail.forStatusAndDetail(codigo, texto)`	Crea la respuesta de error con el código y el detalle.
`problema.setTitle(…)`	Añade el título.

5. Quita la anotación `@ResponseStatus` de las dos excepciones: ahora es el manejador el que decide el código.
6. Reinicia y repite en Postman todas las peticiones de error. Comprueba que cada respuesta tiene el código correcto y un cuerpo que **explica** el problema.

Resumen: qué ocurre en cada caso

Petición	Qué pasa dentro	Respuesta
`GET /api/productos/99`	`buscarOLanzar` lanza `RecursoNoEncontradoException`	404 · "No existe el producto 99"
`DELETE /api/productos/99`	`buscarOLanzar` lanza `RecursoNoEncontradoException`	404 · "No existe el producto 99"
`POST` con el nombre vacío	`validar` lanza `PeticionIncorrectaException`	400 · "El nombre es obligatorio"
`PUT /api/productos/1` con precio -5	`validar` lanza `PeticionIncorrectaException`	400 · "Precio negativo"
`POST` con datos correctos	No se lanza nada	201 y el producto creado

Ampliación: los errores que genera Spring

Algunos errores no los lanzas tú, sino Spring: un JSON mal escrito (400) o un cuerpo que no es JSON (415). Esos errores todavía llegan con otro formato. Añade a `application.properties` la línea `spring.mvc.problemdetails.enabled=true` y repite esas dos peticiones. ¿Qué ha cambiado?

Piensa también: ¿por qué **no** conviene añadir al manejador un método que capture `Exception.class`, es decir, cualquier excepción?

8. Negociación de contenido y configuración

Accept y Content-Type

Un mismo recurso puede tener varias **representaciones** (principio REST). Cliente y servidor acuerdan el formato mediante dos cabeceras. A esto se le llama **negociación de contenido**:

Cabecera	Quién la envía	Qué dice	Si no se puede cumplir
`Accept`	El cliente, en la petición	En qué formato **quiere recibir** la respuesta	406 Not Acceptable
`Content-Type`	Quien envía un cuerpo	En qué formato **está** el cuerpo que envía	415 Unsupported Media Type

En los mapeos se puede restringir qué formatos produce y acepta un endpoint con los atributos `produces` y `consumes`:

Atributos produces y consumes

```
@GetMapping(value = "/{id}",
            produces = { MediaType.APPLICATION_JSON_VALUE,
                         MediaType.APPLICATION_XML_VALUE })

@PostMapping(consumes = MediaType.APPLICATION_JSON_VALUE)
```

Configuración de la ruta base

Todas las rutas de la aplicación cuelgan de la raíz `/`. La propiedad `server.servlet.context-path` añade un prefijo a **toda** la aplicación, algo útil cuando varias aplicaciones comparten el mismo servidor o dominio:

application.properties

```
server.servlet.context-path=/tienda
```

Con esa propiedad, la API pasaría a estar en `http://localhost:8080/tienda/api/productos`. Observa la diferencia con `@RequestMapping`: la propiedad afecta a toda la aplicación, mientras que la anotación afecta solo a un controlador.